

1 22/06/2020

1.1 Traccia da 12 punti

1.1.1 3pt

Siano date 2 matrici di interi M_1 e M_2 , di dimensioni rispettivamente $r_1 \times c_1$ e $r_2 \times c_2$. Si scriva una funzione `submat` che identifichi la più grande sottomatrice quadrata comune di M_1 e M_2 e le sue dimensioni. Tale sottomatrice deve essere memorizzata in una terza matrice `res`, allocata della dimensione opportuna. Il prototipo della funzione `submat` sia:

```
int **subMat(int **M1, int r1, int c1, int **M2, int r2, int c2, int *dim);
```

Esempio

$$M_1 = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 1 & 1 & 3 & 4 \\ 1 & 2 & 3 & 4 \\ 1 & 2 & 4 & 4 \end{pmatrix} \quad M_2 = \begin{pmatrix} 1 & 9 & 3 & 5 \\ 9 & 2 & 3 & 5 \\ 1 & 2 & 4 & 5 \end{pmatrix} \quad res = \begin{pmatrix} 2 & 3 \\ 2 & 4 \end{pmatrix}$$

1.1.2 3pt

Siano dati 2 vettori di interi (positivi, negativi o nulli) v_1 e v_2 di lunghezza rispettivamente d_1 e d_2 . Si scriva una funzione `prodCart` che generi una lista concatenata L contenente il risultato del prodotto cartesiano dei due vettori. La lista sia ordinata in fase di costruzione per prodotto crescente degli elementi di ogni coppia. Il prototipo della funzione `prodCart` sia:

```
list_t prodCart(int *v1, int d1, int *v2, int d2);
```

Si definisca la lista come ADT di I classe e il tipo nodo come quasi ADT. Si indichi esplicitamente in quale modulo/file appare la definizione dei tipi proposti. **È vietato l'uso di funzioni di libreria.**

Esempio

$$v_1 = (1, 2, 3)$$

$$v_2 = (3, 4, 5, 6)$$

$$L = (1, 3), (1, 4), (1, 5), (2, 3), (1, 6), (2, 4), (3, 3), (2, 5), (3, 4), (2, 6), (3, 5), (3, 6)$$

1.1.3 6pt

Sia dato un set di N oggetti, numerati da 1 a N . Ogni oggetto è caratterizzato da un suo nome e da una quaterna di interi positivi o nulli: **costo**, **valore**, **tipo**, **quantità**. Si scriva una funzione ricorsiva in grado di identificare un insieme di oggetti tale da rispettare i seguenti vincoli:

- Costo complessivo al più C , dove C è un intero passato come parametro
- Insieme composto solo da oggetti tipologie diverse
- Si termini l'esecuzione non appena trovata una soluzione valida
- Si giustifichi la scelta del modello combinatorio adottato
- Si giustifichi la scelta del/dei criteri di pruning adottati, o il motivo della loro assenza.

Il prototipo della funzione ricorsiva sia il seguente:

```
int solve_r(item_t *v, int n, int *sol, int pos, int tot_c, int tot_v, ...);
```

1.2 Traccia da 18 punti

1.2.1 Breve introduzione e definizioni

Una matrice rettangolare di dimensione $R \times C$ rappresenta uno schema (a griglia) per un gioco in stile “parole crociate” (o “cruciverba”), che comprende:

- Caselle nere, in cui non è possibile scrivere nulla
- Caselle bianche, in cui vanno scritti caratteri alfabetici (si supponga, per semplicità, di usare solamente lettere maiuscole).

Una volta completata, la griglia deve essere tale per cui ogni sequenza di almeno due caselle bianche (sia in orizzontale che in verticale) deve contenere parole “valide”, non prolungabili (a sinistra/destra le orizzontali, sopra/sotto le verticali). Il gioco consiste nel collocare nella griglia (in orizzontale e in verticale) parole, scelte da un elenco dato, in modo tale da non lasciare caselle bianche vuote. Si noti che:

- Ogni parola orizzontale dovrà iniziare in una casella bianca priva di una casella bianca adiacente a sinistra, e terminare in una casella bianca priva di casella bianca adiacente a destra. Ragionamento analogo/duale va fatto per le parole in verticale.

- Gli incroci tra parole orizzontali e verticali vanno rispettati, cioè una casella bianca può appartenere sia a una parola verticale che orizzontale.

Esempio

H		D			P								M	
E		A	L	G	O	R	I	T	M	I		D	F	S
A		T			I						I		T	
P	R	O	G	R	A	M	M	A	Z	I	O	N	E	
S			R							T		A		
O		H	A	S	H	T	A	B	L	E		M		
R			F			R		S		M	A	I	N	
T	I	P	O			L	I	S	T	A			C	
		Q				E				S	T	A	C	K

Si vogliono risolvere in C i seguenti problemi:

- Definire una opportuna struttura dati in grado di rappresentare lo schema di gioco, utilizzabile per risolvere i problemi che seguono
- Verificare che uno schema già completato sia compatibile con un dato elenco di parole.
- Dato uno schema libero e un elenco di parole, trovare (se esiste) un possibile completamento dello schema mediante la selezione di un sottoinsieme delle parole.

1.2.2 Struttura dati e formato file

Tra i vari formati possibili per rappresentare uno schema, si è scelto quello di elencare in modo esplicito collocazione e lunghezza delle parole orizzontali e verticali. Date queste, le caselle nere, se necessario, possono essere determinate in modo univoco. Lo schema è rappresentato in un file che contiene:

- Prima riga, due interi **R** e **C** che rappresentano le dimensioni ed un intero **N** che rappresenta il numero di parole

- La collocazione delle parole orizzontali e verticali nello schema, una per riga, nel formato `<lunghezza> <r> <c> <direzione>`, dove `<lunghezza>` è la lunghezza della parola, `<r>` e `<c>` sono gli indici della casella di inizio e `<direzione>` è uno tra i caratteri '0' (per orizzontale) oppure 'V' (per verticale).

Esempio

```
15 9 18 // Schema 15x9 con 18 punti di inizio per le parole
8 0 0 V // Parola verticale iniziante in posizione [0,0] di lunghezza 8 caratteri
4 0 2 V
4 0 6 V
...
5 7 5 0 // Parola orizzontale iniziante in posizione [7,5] di lunghezza 5 caratteri
2 7 2 V
5 8 10 0
```

Per completare lo schema, è fornito un insieme di parole. Le parole sono riportate in un secondo file con il seguente formato:

- Sulla prima riga appare il numero P di parole
- Seguono P righe riportanti una parola ognuna.

Esempio

```
50
ALGORITMI
PROGRAMMAZIONE
DIJKSTRA
BST
UNIONFIND
...
```

Le parole hanno lunghezza massima 20 caratteri e non si ripetono nel file. Si definiscano due strutture dati:

- una (con wrapper `struct schema`) in grado di gestire lo schema, cioè le collocazioni (per le parole) nella griglia, sia orizzontali che verticali, ognuna caratterizzata da casella di inizio e da lunghezza.
- una (con wrapper `struct parole`) adatta a immagazzinare gli elenchi delle parole ammesse, organizzate per lunghezza, in modo tale da poter accedere con costo $O(1)$ all'elenco delle parole di una data lunghezza

Si scrivano le funzioni, per l'acquisizione di tali strutture dati dai file.

1.2.3 Funzione di verifica

Si scriva una funzione che, ricevuti come parametri uno schema, un elenco di parole e una matrice di caratteri (di dimensione compatibile con lo schema) contenente uno schema di cruciverba completato (il contenuto delle caselle nere è trascurabile, in quanto vanno solo controllate le parole), verifichi se le parole orizzontali e verticali presenti nello schema sono compatibili con l'elenco delle parole. Il prototipo della funzione sia:

```
int verificaSchema(schema *s, parole *p, char **m);
```

1.2.4 Problema di ricerca

Scopo del gioco è di posizionare opportunamente un sottoinsieme delle parole nella griglia completandola. Le parole non possono comparire più di una volta nella griglia. È sufficiente individuare la prima soluzione valida trovata, ammesso che esista. Individuata la prima soluzione, la funzione termina.

- Si dica a quale schema di funzione ricorsiva si fa riferimento e perché
- Si definisca la struttura dati usata per rappresentare la soluzione struct soluzione
- Si scriva la funzione wrapper di invocazione della funzione ricorsiva di ricerca

```
void solve(schema *s, parole *p);
```

- Si scriva la funzione ricorsiva di ricerca che trova (se esiste) un elenco di parole in grado di completare lo schema

```
void solve_r(schema *s, parole *p, soluzione *sol, int pos, ..., int *trovato);
```

- Si scriva una funzione che data una soluzione, verifichi se questa rispetta i vincoli (ad esempio, una parola verticale e una orizzontale devono avere lo stesso carattere al loro incrocio) oppure no. Una soluzione è data dalle parole selezionate per completare lo schema; si può trattare di soluzione parziale o completa, a seconda che si richiami la funzione a scopo di pruning oppure in un caso terminale della ricorsione. Completare opportunamente il seguente prototipo e l'implementazione della funzione sulla base della scelta di utilizzo effettuata.

```
int checkSol(schema *s, soluzione *sol, ...);
```
